

```
[[2, 8],
 [5, 1]],

[[3, 9],
 [6, 2]]])
```

sum(a [, axis][, dtype][, out][, keepdims]): This returns the sum of elements along the given axis. In the option list is an ndarray into which the result should be written. *keepdims* is a Boolean value, which if set to 'True', will keep the axis which is reduced a dimension with size one in the result. An example of a 3D array is given below. In this case, the axis takes values from 0 to 2.

```
>>>a=n.array(((1,2,3),(4,5,6)),((7,8,9),(0,1,2)))
>>>a
array([[[1, 2, 3],
        [4, 5, 6]],

       [[7, 8, 9],
        [0, 1, 2]])]
>>>n.sum(a,axis=0)
array([[ 8, 10, 12],
        [ 4,  6,  8]])
>>>n.sum(a,axis=1)
array([[ 5,  7,  9],
        [ 7,  9, 11]])

>>> n.sum(a,axis=2,keepdims=True)
array([[[ 6],
         [15]],

       [[24],
         [ 3]]])
```

prod(a [, axis][, dtype][, out][, keepdims]): This returns the product of elements along the given axis.

There are functions like *argmax*, *min*, *argmin*, *ptp*, *clip*, *conj*, *round*, *trace*, *cumsum*, *mean*, *var*, *std*, *cumprod*, *all* and *any*, which make scientific computations easier. There are functions for array conversion, shape manipulation, item selection and manipulation too. If one wishes to dig deep, please visit the official SciPy site.

Operations on arrays

Arithmetic operations: Arithmetic operators like '+', '-', '*', '/' and '%' can be applied directly on NumPy arrays. It is to be noted that all operations are element-wise operations. The result of 2D array multiplication is shown below:

```
>>>c=n.array([[1,2],[3,4]])
>>>d=n.array([[1,3],[2,1]])
>>>c*d
array([[1, 6],
```

```
[6, 4]])
```

If you wish to perform matrix multiplication, use the function *dot()* as shown below or generate matrices using the matrix function and use the '**' operator on them.

```
>>>c=n.array([[1,2],[3,4]])
>>>d=n.array([[1,3],[2,1]])
>>> c
array([[1, 2],
        [3, 4]])
>>> d
array([[1, 3],
        [2, 1]])
>>> n.dot(c,d)
array([[ 5,  5],
        [11, 13]])
```

Relational operations: NumPy allows one to compare two arrays using relational operators. The result will be a Boolean array, i.e., an element in a resultant array is set to 'True' only if the condition is satisfied. An example is shown below:

```
>>>c=n.array([[1,2],[3,4]])
>>>d=n.array([[1,3],[2,1]])
>>>c
array([[1, 2],
        [3, 4]])
>>>d
array([[1, 3],
        [2, 1]])
>>>c=d
array([[ True, False],
        [False, False]], dtype=bool)
```

Logical operations: Logical operations can be performed on arrays using built-in functions supported by NumPy. Functions like *logical_or()*, *logical_not()*, *logical_and()*, etc can be used for this purpose. The code segment given below shows the results of the XOR operation.

```
>>>c=n.array([[0,1],[2,3]])
>>>d=n.array([[1,3],[2,0]])
>>>n.logical_xor(c,d)
array([[ True, False],
        [False,  True]], dtype=bool)
```

Indexing and slicing arrays

The NumPy array index starts at 0. Let 'a' be a 2D array. 'a[i][j]' represents the (j+1)th element in the (i+1)th row. Equivalently, you can write it as 'a[i,j]'. 'a[3,:]' represents all elements in the 4th row. 'a[:i+2,:]' represents all the elements in the (i+1)th row to the (i+3)rd row.

I am now going to explain an attractive feature of NumPy